Advanced Programming (OOP) LAB

Project Title: Student Attendance Management System Using OOP in Java (Console-Based)

Student Name: Anindya Saha

Roll Number: 81

Instructors: Dr. Swagatam Basu, Dr. Susovan Jana, Dr. Soumadip Biswas

Semester: Spring 2026

2. Problem Statement

Attendance management is a fundamental task in educational institutions. Traditional methods involve maintaining registers manually, which is both time-consuming and prone to human error.

Teachers often face difficulty in calculating attendance percentages, maintaining records, and retrieving data efficiently.

Manual systems lack scalability, accuracy, and quick accessibility, especially when the number of students increases.

To overcome these challenges, a console-based Student Attendance Management System is developed using Object-Oriented Programming principles in Java.

The system automates student registration, attendance marking, and percentage calculation, ensuring better data organization and improved efficiency.

3. System Design

The system follows a layered architecture approach separating user interface, business logic, and data handling.

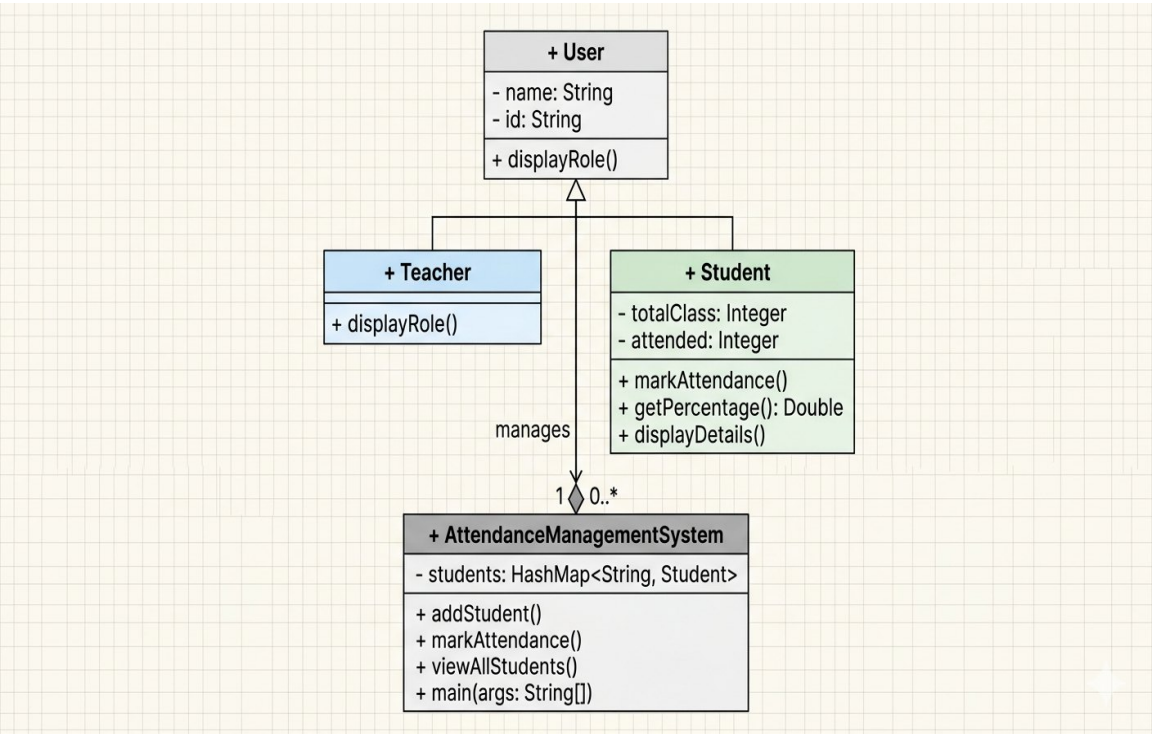
UI Layer: Handles interaction through menu-driven console.

Service Layer: Contains business logic like attendance calculation.

Repository Layer: Manages data storage using HashMap.

Classes: User (abstract), Student, Teacher, AttendanceService, StudentRepository.

Interaction Flow: User in put validation processing output display.



4. OOP Concepts Demonstrated

Abstraction is implemented using an abstract class 'User' which defines the structure for all users.
Inheritance allows Student and Teacher classes to reuse properties from User class.
Polymorphism is demonstrated through method overriding, enabling different behaviors for displayRole().
Exception Handling ensures the system remains stable during invalid inputs or operations.
Collections such as HashMap are used to store and manage student data efficiently.

```
public abstract class User {
   protected String name;
   protected String id;
   public User(String name, String id) {
      this.name = name;
      this.id = id;
   }
   public abstract void displayRole();
}
```

Explanation:
The abstract class hides implementation details and provides only essential structure. It ensures that all subclasses must define their own version of displayRole(). This improves code organization and enforces a consistent design across different user types.

```
public class Student extends User {
   private int totalClasses;
   private int attendedClasses;

   public Student(String name, String id) {
      super(name, id);
   }
}
```

Explanation:
Inheritance allows reuse of common attributes (name, id) and methods from the parent class. It reduces code duplication and establishes a clear hierarchical relationship, making the system easier to extend and maintain.

```
public void markAttendance(String id, boolean present) throws StudentNotFoundException {
   Student s = repo.getStudent(id);

   if (s == null) {
      throw new StudentNotFoundException("Student not found!");
   }

   s.markAttendance(present);
}
```

Explanation:
This ensures that invalid operations (like marking attendance for a non-existing student) are properly handled. Instead of crashing, the system throws a meaningful exception which is caught in the UI layer. This improves system robustness, reliability, and user experience.

5. Implementation Highlights with Detailed Code Snippets

Snippet 1: Abstract Class (Abstraction)

```
public abstract class User { protected String name; protected String id; public
abstract void
displayRole(); }
```

Explanation: This abstract class defines a common structure for all users. It ensures that
subclasses must implement displayRole(), achieving abstraction by hiding implementation details.

Snippet 2: Inheritance (Student Class)

```
public class Student extends User { private int totalClasses; private int
attendedClasses; }
```

Explanation: The Student class inherits properties from User, promoting code reuse and logical
hierarchy.

Snippet 3: Polymorphism

```
@Override public void displayRole() { System.out.println('Role: Student'); }
```

Explanation: Method overriding allows different behavior for the same method, demonstrating
runtime polymorphism.

Snippet 4: Singleton Pattern

```
private static AttendanceService instance; public static AttendanceService
getInstance(){
if(instance==null) instance=new AttendanceService(); return instance; }
```

Explanation: Ensures only one instance of AttendanceService exists, controlling access to business
logic.

Snippet 5: HashMap Usage (Collections)

```
private HashMap students = new HashMap<>();
```

Explanation: HashMap provides fast data retrieval using key-value pairs, improving efficiency.

Snippet 6: Exception Handling

```
if(s == null){ throw new StudentNotFoundException('Student not found'); }
```

Explanation: Custom exception improves clarity and prevents invalid operations.

Snippet 7: Input Handling

```
public static int getInt(String msg){ return Integer.parseInt(sc.nextLine()); }
```

Explanation: Converts user input into integer while allowing exception handling for invalid inputs.

6. Testing & Error Handling

Test Case	Input	Expected Output
Add Student	Valid ID	Student added successfully
Duplicate Entry	Same ID	Error message displayed
Mark Attendance	Valid ID	Attendance updated
Invalid Student	Wrong ID	Error message shown
View Records	Multiple entries	All data displayed

Edge cases such as empty records and division by zero are handled effectively.

Failure scenarios like invalid inputs and incorrect choices are managed using exception handling.

Handling Strategy:
Try-catch blocks are used to catch runtime exceptions. Custom exceptions provide meaningful error messages.
Example Flow:
User enters invalid ID system checks repository exception thrown caught in UI message displayed

7. Git Workflow

The project development was carried out incrementally with commits representing each stage of development.

Add files via upload anindyasaha2310-ops authored 3 days ago	Verified	...
Create user.java anindyasaha2310-ops authored 3 days ago	Verified	...
Create Student anindyasaha2310-ops authored 3 days ago	Verified	...
Create Teacher anindyasaha2310-ops authored 3 days ago	Verified	...
Create StudentRepository anindyasaha2310-ops authored 3 days ago	Verified	...
Create StudentNotFoundException anindyasaha2310-ops authored 3 days ago	Verified	...
Create AttendanceService anindyasaha2310-ops authored 3 days ago	Verified	...
Create InputUtil anindyasaha2310-ops authored 3 days ago	Verified	...
Create MainApp anindyasaha2310-ops authored 3 days ago	Verified	...
Create attendance-system anindyasaha2310-ops authored 3 days ago	Verified	...
Delete src/model directory anindyasaha2310-ops authored 3 days ago	Verified	...
Delete src/attendance-system anindyasaha2310-ops authored 3 days ago	Verified	...
Create User anindyasaha2310-ops authored 3 days ago	Verified	...

8. Conclusion & Future Scope

The project successfully demonstrates the practical application of Object-Oriented Programming concepts.

It provides an efficient and structured solution for attendance management.

Future Scope: Integration with GUI, database storage, authentication system, and report generation features.

Appendix

PCCCS495 Proposal Review – APPROVED_MINOR

S soumadip.biswas@ieminternation... 4 Mar
To You

Dear Anindya Saha ,

Project: Student Attendance Management System Using OOP in Java(Console based)
Decision: APPROVED_MINOR
Score: 85

Strengths:
The proposal clearly identifies a common, real-world problem that is well-suited for an introductory OOP project. The target audience is well-defined, and the proposed features are practical, achievable, and appropriate for a console-based system. Explicit mention of key OOP concepts like Inheritance, Polymorphism, Exception Handling, and Collections demonstrates a good understanding of the course objectives. The architecture suggests a modular design with separate classes, which is excellent for applying and showcasing OOP principles. It's a solid proposal for demonstrating fundamental Java OOP